

WEEK – 1, 2

Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

JAVA BANK MANAGEMENT SYSTEM

Object-Oriented Programming Practical Record

1. Steps for Execution

1. Open VS Code, Eclipse or IntelliJ IDEA with JDK installed.
2. Create a Java file named Bank.java.
3. Enter the Java source code and save the file.
4. Compile the program using `javac Bank.java`.
5. Run the program using `java Bank`.
6. The program creates SavingsAccount and CurrentAccount objects.
7. Perform deposit, withdrawal, interest calculation and transfer operations.
8. Display the final account details.

2. Algorithm

1. Start the program.
2. Create a SavingsAccount object with account number, balance and interest rate.
3. Create a CurrentAccount object with account number, balance and overdraft limit.
4. Display the initial details of both accounts.
5. Deposit an amount into the savings account.
6. Withdraw an amount from the current account.
7. Calculate interest on the savings account.
8. Transfer an amount from the savings account to the current account.
9. Perform another withdrawal from the current account.
10. Display the final account details.
11. Stop the program.

3. Explanation of the Program

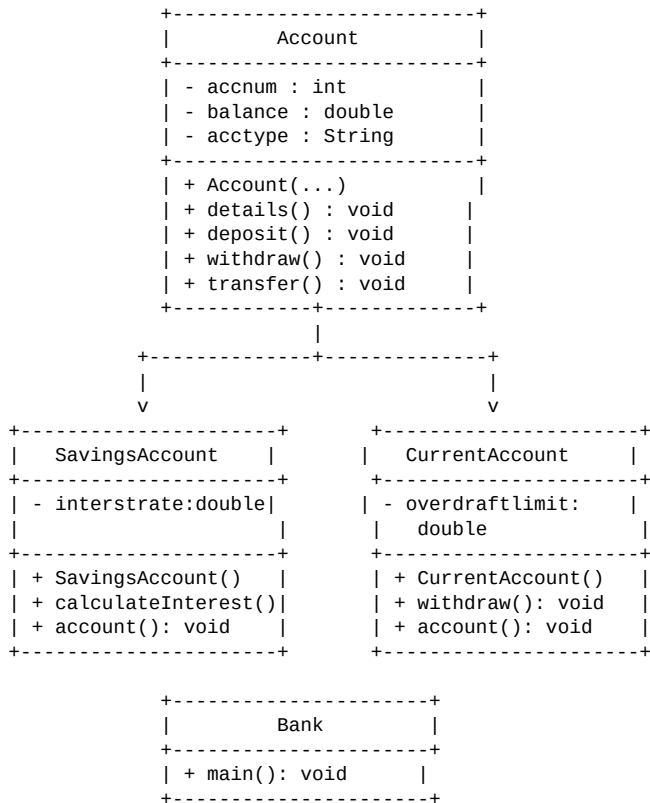
Account class: Parent class containing account number, balance, account type and common operations such as deposit, withdraw, details and transfer.

SavingsAccount class: Extends Account and stores the interest rate. The `calculateInterest()` method calculates and adds interest.

CurrentAccount class: Extends Account and overrides `withdraw()` to allow withdrawal using the overdraft limit.

Bank class: Contains `main()` and creates SavingsAccount and CurrentAccount objects to perform operations.

4. UML Class Diagram



5. Java Source Code

```
class Account {
    int accnum;
    double balance;
    String acctype;

    Account(int n, double b, String type) {
        accnum = n;
        balance = b;
        acctype = type;
    }

    void details() {
        System.out.println("Account Number : " + accnum);
        System.out.println("Balance : " + balance);
        System.out.println("Account Type : " + acctype);
    }

    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited : " + amount);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }

    void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn : " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
    }

    void transfer(Account receiver, double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            receiver.balance += amount;
            System.out.println("Transferred : " + amount);
            System.out.println("To Account : " + receiver.accnum);
        }
    }
}
```

```

        } else {
            System.out.println("Transfer failed");
        }
    }
}

class SavingsAccount extends Account {
    double interstrate;

    SavingsAccount(int n, double b, double rate) {
        super(n, b, "Savings Account");
        interstrate = rate;
    }

    void calculateInterest() {
        double interest = balance * interstrate / 100;
        balance += interest;
        System.out.println("Interest Added : " + interest);
    }

    void account() {
        System.out.println("Account Number : " + accnum);
        System.out.println("Balance      : " + balance);
        System.out.println("Interest Rate : " + interstrate);
    }
}

class CurrentAccount extends Account {
    double overdraftlimit;

    CurrentAccount(int n, double b, double limit) {
        super(n, b, "Current Account");
        overdraftlimit = limit;
    }

    @Override
    void withdraw(double amount) {
        if (amount > 0 && amount <= balance + overdraftlimit) {
            balance -= amount;
            System.out.println("Withdrawn : " + amount);
        } else {
            System.out.println("Overdraft limit exceeded");
        }
    }

    void account() {
        System.out.println("Account Number : " + accnum);
        System.out.println("Balance      : " + balance);
        System.out.println("Overdraft Limit: " + overdraftlimit);
    }
}

public class Bank {
    public static void main(String[] args) {
        SavingsAccount s = new SavingsAccount(3010, 5000, 6);
        CurrentAccount c = new CurrentAccount(4020, 7000, 2000);

        System.out.println("SAVINGS ACCOUNT");
        s.account();

        System.out.println("\nCURRENT ACCOUNT");
        c.account();

        System.out.println("\nDEPOSIT");
        s.deposit(1500);

        System.out.println("\nWITHDRAW");
        c.withdraw(2500);

        System.out.println("\nINTEREST");
        s.calculateInterest();

        System.out.println("\nTRANSFER");
        s.transfer(c, 1000);

        System.out.println("\nWITHDRAW");
        c.withdraw(6000);

        System.out.println("\nFINAL ACCOUNT DETAILS");
        System.out.println("\nSavings Account");
        s.account();
    }
}

```

```
        System.out.println("\nCurrent Account");  
        c.account();  
    }  
}
```

6. Expected Final Balances

Savings Account: 5890.0

Current Account: 500.0